

SQL DBA CHAMPS

Learning Path Series

Amazon DynamoDB

From Basics to Advanced — Step by Step

Data Model · Capacity · Indexes · Single-Table Design · Streams · Real-Time Scenarios

A hands-on teaching guide for SQL Server DBAs moving into AWS NoSQL

Prepared by Praveen Madupu

Table of Contents

Table of Contents	2
Part 1 — Foundations: What DynamoDB Is and the NoSQL Mindset	3
1.1 What is Amazon DynamoDB?	3
1.2 Relational vs DynamoDB mindset.....	3
1.3 When to use DynamoDB — and when not to	3
Part 2 — Core Data Model & Keys.....	4
2.1 Tables, items, attributes	4
2.2 Primary keys — the most important decision	4
2.3 Attribute data types.....	4
Part 3 — Capacity Modes & Throughput.....	5
3.1 On-Demand vs Provisioned	5
3.2 Quick capacity math	5
Part 4 — Reading & Writing Data	6
4.1 Core item operations.....	6
4.2 Query vs Scan (learn this well).....	6
4.3 Consistency, conditional writes & transactions.....	6
4.4 PartiQL — a SQL-like option	6
Part 5 — Secondary Indexes.....	8
5.1 GSI vs LSI	8
5.2 Powerful index patterns	8
Part 6 — Advanced Data Modeling (Single-Table Design).....	9
6.1 Design access-patterns-first	9
6.2 Single-table design.....	9
6.3 Many-to-many: the adjacency list pattern	9
6.4 Write sharding for hot keys	9
Part 7 — Advanced Features	10
7.1 DynamoDB Streams (change data capture)	10
7.2 Global Tables (multi-region, active-active).....	10
7.3 DAX — DynamoDB Accelerator	10
7.4 TTL (Time To Live)	10
7.5 Backups, PITR & data movement	10
Part 8 — Security & Operations	11
8.1 Security.....	11
8.2 Monitoring & cost.....	11
Part 9 — Real-Time Scenarios	12
Part 10 — Quick Reference	14
10.1 SQL Server → DynamoDB mental map.....	14
10.2 Rules & numbers worth memorising.....	14

Part 1 — Foundations: What DynamoDB Is and the NoSQL Mindset

LEARNING GOAL

Understand what DynamoDB is, how the NoSQL mindset differs from relational thinking, and when to use it (and when not to).

1.1 What is Amazon DynamoDB?

DynamoDB is a **fully managed, serverless, NoSQL key-value and document database**. It delivers single-digit-millisecond latency at virtually any scale, with no servers to patch, no storage to provision, and automatic replication across multiple Availability Zones for durability and availability.

- **Serverless** — you create tables, not clusters. AWS handles capacity, replication and patching.
- **Key-value + document** — retrieve an item by its key, or store rich nested JSON-like documents.
- **Predictable performance** — latency stays flat whether the table holds a thousand items or a trillion.

SQL DBA TRANSLATION

Coming from SQL Server: forget tables with fixed columns, JOINS, and a query planner. In DynamoDB you design around access patterns, there are no JOINS, and you retrieve data by keys you chose in advance. The database is fast because it does less thinking at query time — you do that thinking during design.

1.2 Relational vs DynamoDB mindset

Relational (SQL Server)	DynamoDB (NoSQL)
Normalise, then query flexibly with JOINS	Model for known access patterns; no JOINS
Schema defined up front (all columns)	Only the key attributes are fixed; items vary
Query planner optimises at runtime	You pre-plan retrieval via keys and indexes
Scale up (bigger box) / read replicas	Scales out horizontally, automatically
Ad-hoc queries are cheap and normal	Ad-hoc scans are expensive — avoid them

1.3 When to use DynamoDB — and when not to

- **Great fit:** high-scale web/mobile backends, session stores, shopping carts, gaming leaderboards, IoT/time-series ingestion, event sourcing — workloads with well-known access patterns and huge throughput.
- **Poor fit:** complex ad-hoc analytics, heavy multi-table JOINS, reporting with unpredictable query shapes — those still suit a relational or analytics engine.

Part 2 — Core Data Model & Keys

LEARNING GOAL

Master tables, items, attributes, data types, and — most importantly — primary keys, which control where data physically lives.

2.1 Tables, items, attributes

- **Table** — a collection of items (loosely, a "table" of rows).
- **Item** — a single record, up to 400 KB, identified by its primary key (like a row).
- **Attribute** — a name/value pair on an item (like a column) — but items need not share the same attributes.

2.2 Primary keys — the most important decision

Every table has a primary key, and it comes in two shapes:

Key type	Made of	Behaviour
Simple (partition key only)	Partition key (a.k.a. hash key)	Each key must be unique; item located by hashing the key
Composite (partition + sort key)	Partition key + sort key (range key)	Many items share a partition key; sorted by sort key — enables ranges

CONCEPT — How the partition key decides physical placement

DynamoDB hashes the partition key to pick a physical partition. Items with the same partition key live together (an 'item collection') and are stored sorted by the sort key. A good partition key is high-cardinality and evenly accessed — that's what keeps performance flat.

WATCH OUT — The hot partition trap

If most traffic hits one partition-key value (e.g. a single 'status = ACTIVE' key), that partition becomes a bottleneck. Design keys so load spreads evenly; if needed, add a suffix to shard writes across many partition-key values.

2.3 Attribute data types

- **Scalar** — String (S), Number (N), Binary (B), Boolean (BOOL), Null.
- **Document** — Map (M, nested object) and List (L, ordered array) — store JSON-like structures.
- **Set** — String Set (SS), Number Set (NS), Binary Set (BS) — unordered unique values.

Part 3 — Capacity Modes & Throughput

LEARNING GOAL

Choose between On-Demand and Provisioned capacity, and compute read/write costs in RCUs and WCUs.

3.1 On-Demand vs Provisioned

	On-Demand	Provisioned
You set	Nothing — it scales automatically	RCUs and WCUs (with optional Auto Scaling)
Scaling	Instant, handles spikes	Scales to a target utilisation via Auto Scaling
Cost model	Pay per request	Pay for provisioned capacity per hour
Best for	Unpredictable/spiky or new workloads	Steady, predictable traffic (cheaper at scale)

CONCEPT — RCU and WCU — the units of throughput

1 WCU = one write per second for an item up to 1 KB (larger items cost proportionally more).

1 RCU = one strongly consistent read per second up to 4 KB, OR two eventually consistent reads per second up to 4 KB.

Transactional reads/writes consume double the units.

3.2 Quick capacity math

Example: you need 100 strongly consistent reads/sec of 8 KB items, and 50 writes/sec of 2 KB items.

Reads: item 8 KB -> $\text{ceil}(8/4) = 2$ RCU per read
100 reads/sec $\times 2 = 200$ RCU (strongly consistent)

Writes: item 2 KB -> $\text{ceil}(2/1) = 2$ WCU per write
50 writes/sec $\times 2 = 100$ WCU

KEY TAKEAWAY

Eventually consistent reads are half the cost of strongly consistent ones. If your app tolerates data that may be a fraction of a second stale, you halve your read capacity bill.

Part 4 — Reading & Writing Data

LEARNING GOAL

Learn the core operations, the crucial Query-vs-Scan distinction, consistency, conditional writes, transactions, and PartiQL.

4.1 Core item operations

Operation	Does	SQL analogy
PutItem	Create or fully replace an item	INSERT / UPSERT
GetItem	Fetch one item by primary key	SELECT ... WHERE PK =
UpdateItem	Modify attributes of an item	UPDATE
DeleteItem	Remove an item	DELETE
BatchGetItem / BatchWriteItem	Up to 100 gets / 25 writes per call	Bulk operations

4.2 Query vs Scan (learn this well)

- **Query** — retrieves items by partition key (plus an optional sort-key condition). Fast and efficient; only reads matching items. This is how you should read data.
- **Scan** — reads the entire table and filters afterwards. Slow and expensive at scale; use only for admin tasks or small tables.

WATCH OUT

A Scan reads every item and consumes capacity for all of them before filtering. On a large table this is costly and slow. If you find yourself needing a Scan for a routine access pattern, your key/index design is probably wrong.

Each Query or Scan returns up to 1 MB per call; you paginate with the returned LastEvaluatedKey for more.

```
# Query: all orders for a customer, newest first (CLI)
aws dynamodb query \
  --table-name Orders \
  --key-condition-expression 'PK = :c AND begins_with(SK, :o)' \
  --expression-attribute-values \
    '{"c":{"S":"CUST#1001"},"o":{"S":"ORDER#"}}' \
  --scan-index-forward false
```

4.3 Consistency, conditional writes & transactions

- **Eventually consistent (default)** — cheapest; a read may briefly miss the newest write.
- **Strongly consistent** — always returns the latest committed data (not available on GSIs).
- **Conditional writes** — only write if a condition holds (e.g. `attribute_not_exists(PK)`) — enables optimistic locking and idempotency.
- **Transactions** — `TransactWriteItems` / `TransactGetItems` give all-or-nothing ACID across up to 100 items.

4.4 PartiQL — a SQL-like option

DynamoDB supports PartiQL, a SQL-compatible query language, for those who prefer familiar syntax — though it still runs under DynamoDB's key/index rules underneath.

```
-- PartiQL SELECT (still uses the key to stay efficient)
SELECT * FROM Orders WHERE PK = 'CUST#1001' AND begins_with(SK, 'ORDER#');
```

SQL DBA TRANSLATION

PartiQL looks like SQL, but it is not a relational engine — a PartiQL statement without a key condition becomes a full Scan. Familiar syntax does not remove the need to design for access patterns.

Part 5 — Secondary Indexes

LEARNING GOAL

Add extra access patterns with Global and Local Secondary Indexes, and know exactly when to use each.

A primary key gives you one way to find data. Secondary indexes let you query by other attributes.

5.1 GSI vs LSI

	Global Secondary Index (GSI)	Local Secondary Index (LSI)
Partition key	Any attribute (different from table)	Same as the table's partition key
Sort key	Any attribute (optional)	A different attribute
When created	Any time	Only at table creation
Capacity	Its own RCU/WCU	Shares the table's capacity
Consistency	Eventually consistent only	Strong consistency available
Limit	Up to 20 per table (default)	Up to 5 per table

KEY TAKEAWAY

Default to GSIs — they're flexible and can be added later. Reach for an LSI only when you need strongly consistent reads on an alternate sort key within the same partition, and only if you can define it at table-creation time.

5.2 Powerful index patterns

- **Sparse index** — only items that have the indexed attribute appear in the index. Great for querying a small subset (e.g. only 'open' tickets).
- **Index overloading** — reuse one GSI to serve multiple access patterns by storing generic key attributes (GSI1PK / GSI1SK) whose meaning varies by item type.
- **Projection** — choose which attributes copy into the index (KEYS_ONLY, INCLUDE, ALL) to balance query convenience against storage/write cost.

Part 6 — Advanced Data Modeling (Single-Table Design)

LEARNING GOAL

Learn the design philosophy that trips up most relational veterans: model access patterns first, and often store many entity types in one table.

6.1 Design access-patterns-first

In DynamoDB you do not design a normalized schema and then query it. You list every access pattern the application needs, then design keys and indexes so each pattern is a single efficient Query.

1. List all access patterns (e.g. get customer, list customer's orders, get order items, list products by category).
2. Design primary key + GSIs so each pattern maps to one Query on a key.
3. Denormalize deliberately — duplicate data where it makes reads a single call.

6.2 Single-table design

A common advanced pattern stores multiple entity types (customers, orders, order-items) in one table, using generic key names and prefixes so related items share a partition and come back together.

One table, generic keys (PK, SK), many entity types:

PK	SK	attributes
CUST#1001	PROFILE	name, email, tier
CUST#1001	ORDER#2026-07-01	total, status
CUST#1001	ORDER#2026-07-05	total, status
ORDER#5567	ITEM#SKU-42	qty, price
ORDER#5567	ITEM#SKU-88	qty, price

Query PK='CUST#1001' with begins_with(SK, 'ORDER#')
-> all of that customer's orders in ONE call.

SQL DBA TRANSLATION

This feels wrong to a relational eye — mixed 'row types' in one table, duplicated data, no JOINS. But it is deliberate: co-locating related items lets one Query return a whole object graph at single-digit-millisecond latency, which JOINS can't match at DynamoDB's scale.

6.3 Many-to-many: the adjacency list pattern

Model many-to-many relationships (e.g. users and groups) by storing relationship items and using a GSI to traverse them in the reverse direction — the adjacency-list pattern — instead of a join table plus JOINS.

6.4 Write sharding for hot keys

If one logical partition key would be too hot (e.g. a viral item's votes), spread writes across N synthetic keys (ITEM#42#1 ... ITEM#42#N) and aggregate on read. This trades a little read complexity for even write distribution.

Part 7 — Advanced Features

LEARNING GOAL

The capabilities that make DynamoDB a platform: Streams, Global Tables, DAX, TTL, backups, and integrations.

7.1 DynamoDB Streams (change data capture)

Streams capture an ordered log of every item-level change (insert/modify/remove) and retain it for 24 hours. They commonly trigger AWS Lambda to build event-driven pipelines — replication, aggregation, notifications, search indexing.

SQL DBA TRANSLATION

Think of Streams as DynamoDB's change data capture — conceptually similar to CDC / Change Tracking on SQL Server, but it directly drives serverless functions.

7.2 Global Tables (multi-region, active-active)

Global Tables replicate a table across multiple AWS Regions with multi-active writes — applications in each region read and write locally, and DynamoDB replicates changes with typically sub-second lag. Ideal for global low latency and regional resilience.

WATCH OUT

Multi-active writes mean concurrent writes to the same item in different regions can conflict; DynamoDB resolves with last-writer-wins. Design so the same item isn't hot-written in two regions at once.

7.3 DAX — DynamoDB Accelerator

DAX is a fully managed in-memory cache for DynamoDB that cuts read latency from single-digit milliseconds to microseconds for read-heavy or bursty workloads — with no application rewrite (it's API-compatible).

7.4 TTL (Time To Live)

Set a timestamp attribute as the TTL, and DynamoDB automatically deletes expired items in the background at no extra write cost — perfect for sessions, carts, and ephemeral data. Deletion happens within roughly 48 hours of expiry.

7.5 Backups, PITR & data movement

- **Point-in-time recovery (PITR)** — restore to any second in the last 35 days.
- **On-demand backups** — full backups retained until you delete them, with no performance impact.
- **Import/Export with S3** — bulk load from S3 and export table data to S3 without consuming table capacity.
- **Zero-ETL** — integrate to Amazon OpenSearch (search) and Redshift (analytics) without building pipelines.

Part 8 — Security & Operations

LEARNING GOAL

Secure and observe DynamoDB: IAM fine-grained access, encryption, and monitoring.

8.1 Security

- **IAM access control** — no database users; access is governed by IAM policies. Fine-grained access can even restrict a caller to items whose partition key matches their user ID (leading-key condition).
- **Encryption at rest** — always on, using AWS-owned, AWS-managed, or customer-managed KMS keys.
- **Encryption in transit** — TLS for all API calls.
- **VPC endpoints** — reach DynamoDB privately from your VPC without traversing the public internet.

8.2 Monitoring & cost

- **CloudWatch** — consumed capacity, throttled requests, latency, system errors.
- **Contributor Insights** — identifies the most-accessed keys — your tool for spotting hot partitions.
- **Cost levers** — pick the right capacity mode, use eventually consistent reads, prune GSIs/projections, and apply TTL to shed stale data. Reserved capacity cuts cost for steady provisioned workloads.

Part 9 — Real-Time Scenarios

Applied situations that connect the concepts above to real design decisions.

SCENARIO User session / shopping-cart store

Need: Millions of short-lived sessions with fast key lookups and automatic cleanup.

Design: Table keyed on sessionId (simple partition key). Store cart contents as a document attribute. Add a TTL attribute so expired sessions delete themselves — no cleanup job.

Why: Pure key-value access = single-digit-ms GetItem; TTL removes the need for a purge process.

SCENARIO Gaming leaderboard

Need: Top scores per game, ranked, with fast reads at massive scale.

Design: Partition key = gameId, sort key = score (or use a GSI with score as sort key). Query by gameId and read items in descending sort order for the top N.

Why: The sort key gives ordered results directly; no sorting at query time.

SCENARIO IoT / time-series ingestion

Need: Millions of device readings per minute without hot partitions.

Design: Partition key = deviceId (high cardinality spreads load), sort key = timestamp. For very hot devices, shard the key (deviceId#shard). Use On-Demand for unpredictable ingest, and TTL to age out old readings.

Watch: A timestamp-only partition key would funnel all current writes to one partition — always lead with a high-cardinality key.

SCENARIO Global, low-latency application

Need: Users on three continents need fast local reads/writes and regional resilience.

Design: Enable Global Tables across the relevant regions. Each region serves local traffic; DynamoDB replicates with sub-second lag.

Watch: Avoid writing the same item concurrently from two regions (last-writer-wins resolves conflicts).

SCENARIO Read-heavy app needing microsecond latency

Need: A product catalog hammered by reads, with occasional micro-latency requirements.

Design: Put DAX in front of DynamoDB. Reads hit the in-memory cache (microseconds); it's API-compatible so little/no code change. Provisioned + Auto Scaling handles the write path.

SCENARIO Event-driven pipeline

Need: React to every order change — update search, send notifications, aggregate metrics.

Design: Enable DynamoDB Streams and attach a Lambda function. Each item change invokes the function to fan out to OpenSearch, SNS, and an aggregation table.

Why: Streams give an ordered, exactly-processed change feed without polling.

SCENARIO Re-platform a relational table to DynamoDB

Need: Move a high-traffic Customers/Orders/OrderItems schema off SQL Server for scale.

Design: List access patterns first, then use single-table design: one table with generic PK/SK, prefixes (CUST#, ORDER#, ITEM#), and a GSI or two for reverse lookups. Denormalize so each pattern is one Query.

Reality check: If the workload truly needs ad-hoc joins and flexible reporting, DynamoDB is the wrong target — keep that in a relational/analytics store.

SCENARIO Throttling / hot-partition incident

Symptom: ThrottledRequests climbing though overall capacity looks sufficient.

Diagnose: Use CloudWatch + Contributor Insights to find the over-accessed key. It's usually one hot partition key absorbing disproportionate traffic.

Fix: Introduce write sharding (suffix the key), spread the access pattern, or switch to On-Demand to absorb bursts; re-model the key if the pattern is structural.

Part 10 — Quick Reference

10.1 SQL Server → DynamoDB mental map

SQL Server concept	DynamoDB equivalent
Row	Item
Column	Attribute
Primary key	Partition key (+ optional sort key)
Non-clustered index	Global / Local Secondary Index
JOIN	Denormalize / single-table design (no JOINS)
SELECT ... WHERE key =	GetItem / Query
Table scan	Scan (avoid for routine access)
CDC / Change Tracking	DynamoDB Streams
Always On / geo-replication	Global Tables

10.2 Rules & numbers worth memorising

- Max item size: 400 KB.
- 1 WCU = 1 write/sec up to 1 KB; 1 RCU = 1 strong read/sec (or 2 eventual) up to 4 KB.
- Transactions and strongly consistent ops cost double the capacity.
- Query/Scan return up to 1 MB per call, then paginate.
- GSI: up to 20/table (default), eventually consistent, own capacity. LSI: up to 5/table, created at table creation, can be strongly consistent.
- Streams retain changes 24 hours; PITR covers the last 35 days.
- Prefer Query over Scan; prefer On-Demand for spiky/unknown load.

WATCH OUT — Verify current limits

AWS raises limits and adds features regularly (index counts, throughput ceilings, new integrations). For anything you'll quote as a hard number in an interview or design, confirm it against the current DynamoDB documentation.